TD : relier l'interface à Laravel avec Fetch

Objectif

Vous allez relier l'interface HTML et JavaScript à l'API Laravel construite précédemment. À la fin du TD, la page chargera les documents de PostgreSQL, transmettra le titre et la catégorie du formulaire et affichera un message si une requête échoue.

Le **navigateur** exécute le JavaScript. `fetch` envoie une requête HTTP. **Laravel** traite la requête et travaille avec **PostgreSQL**. La réponse revient en JSON au navigateur, qui met la page à jour.

Cet exercice porte sur les informations des documents : aucun fichier n'est téléversé. La préférence d'affichage compact reste une donnée locale du navigateur.

1. Préparer les deux projets

Côté Laravel

Le projet Laravel doit disposer de l'API CRUD du TD précédent. Utilisez votre travail terminé ou la branche de correction `jour2-api-crud-documents` du dépôt Laravel. Si vous changez de branche, sauvegardez d'abord vos modifications : ne forcez pas le changement et ne supprimez pas votre travail si Git signale un conflit.

Dans un terminal ouvert **à la racine du projet Laravel**, celle qui contient son `compose.yaml`, démarrez les services :

```
docker compose up -d
```

Si l'installation initiale n'a pas encore été faite, terminez le guide Docker du projet Laravel : dépendances Composer, fichier `.env` et clé. Ne recréez pas `.env` et ne régénérez pas la clé sur un projet déjà configuré.

La table `documents` doit être présente dans la base locale de formation. Si ce n'est pas déjà fait, au moment indiqué par le formateur, exécutez :

```
docker compose exec app php artisan migrate
```

Cette commande applique toutes les migrations en attente, y compris celles fournies par Laravel. Vérifiez les routes de l'API :

```
docker compose exec app php artisan route:list --path=api/documents
```

À vérifier : les routes GET, POST, PUT et DELETE des documents apparaissent. Si elles sont absentes, terminez le TD CRUD ou utilisez sa branche de correction.

Côté interface

Ouvrez **le dossier du projet front-end**, distinct du projet Laravel, dans VS Code. Vous devez partir de la branche `jour1-ressources-stockage`, qui contient l'interface et la préférence d'affichage compact.

```
git branch --show-current
```

Si nécessaire, sauvegardez votre travail, puis lancez `git switch jour1-ressources-stockage`. Si Git refuse le changement, demandez au formateur comment conserver vos modifications. La branche `jour2-fetch-documents` servira de correction.

2. Comprendre l'adresse de l'API et CORS

Vous allez ouvrir `index.html` directement dans le navigateur. Le JavaScript appellera Laravel à l'adresse `http://localhost:8088/api/documents`, où le port 8088 correspond au port publié par Docker pour l'application Laravel.

La page locale et l'API n'ont pas la même origine. CORS est le mécanisme qui permet au navigateur de vérifier si l'API autorise cette communication. Pour ses routes API, Laravel dispose déjà d'une configuration CORS par défaut qui accepte les requêtes nécessaires à cet exercice ; il n'y a rien à ajouter ici.

Cette configuration ouverte convient à la démonstration locale sans authentification. Pour une application réelle, il faudra adapter les origines autorisées et la sécurité au contexte. Selon le navigateur, l'ouverture directe en `file://` peut être soumise à des restrictions ; consultez le tableau de dépannage en cas de blocage.

3. Préparer la liste dans la page HTML

Ouvrez `index.html`. Recherchez le compteur initial :

```
              <span id="document-count" class="count-label">3 documents</span>
```

Remplacez-le par :

```
              <span id="document-count" class="count-label">Chargement…</span>
```

Recherchez ensuite le bloc qui commence par `<div id="document-list" class="document-list">` et contient les trois éléments `<article class="document-item">`. Remplacez **tout ce bloc, de son `<div>` ouvrant à son `</div>` fermant**, par :

```
          <p id="list-message" class="text-muted" role="status" aria-live="polite">Chargement des documents…</p>
          <div id="document-list" class="document-list"></div>
```

La liste sera remplie par les données renvoyées par Laravel. Le paragraphe indiquera le chargement, une liste vide ou une erreur. Les trois documents fictifs écrits dans le HTML ne sont plus nécessaires.

4. Adapter le formulaire

Toujours dans `index.html`, recherchez le bloc du champ de fichier, depuis `<div class="mb-4">` jusqu'à son `</div>` fermant :

```
            <div class="mb-4">
              <label class="form-label" for="document-file">Fichier <span class="optional-label">(facultatif)</span>
```

```
</label>
                <input class="form-control" id="document-file" name="file" type="file" accept=".pdf,.doc,.docx">
                <div class="form-text">Démonstration : aucun fichier n’est encore envoyé.</div>
              </div>
```

Remplacez ce bloc par :

```
              <p class="form-text mb-4">Seuls le titre et la catégorie sont enregistrés. L’envoi de fichiers sera étudié plus tard.</p>
```

Enregistrez le fichier. L'API actuelle stocke uniquement `title` et `category` : cette étape ne propose donc pas de sélection de fichier.

5. Comprendre le JavaScript

Dans le code complet de l'étape suivante, repérez ces idées :

- `fetch(apiUrl)` envoie une requête GET. `await` attend la réponse sans bloquer l'ensemble de la page.
- `response.ok` permet de vérifier le statut HTTP. Une réponse 404 ou 500 ne provoque pas automatiquement une erreur JavaScript : il faut la contrôler.
- `await response.json()` transforme la réponse JSON en données JavaScript. La page crée ensuite une ligne pour chaque document avec `createElement` et `textContent`.
- Le formulaire utilise `POST`, l'en-tête `Content-Type: application/json` et `JSON.stringify(...)` pour envoyer les deux champs. L'en-tête `Accept` demande des réponses JSON à Laravel.
- Le bloc `try...catch` affiche un message si l'échange échoue. La réponse 422 indique ici des données refusées par la validation de Laravel. Le bouton est désactivé pendant l'envoi, puis réactivé dans `finally`.

La validation du navigateur aide l'utilisateur ; Laravel vérifie à nouveau les données reçues. Le code emploie `textContent` pour afficher le titre et la catégorie comme du texte.

6. Écrire le code Fetch

Dans VS Code, ouvrez `assets/js/app.js`. Remplacez tout son contenu par le code suivant, puis enregistrez :

```
// Éléments de la page et adresse de l'API.
const form = document.querySelector('#document-form');
const titleInput = document.querySelector('#document-title');
const categoryInput = document.querySelector('#document-category');
const documentList = document.querySelector('#document-list');
const documentCount = document.querySelector('#document-count');
const listMessage = document.querySelector('#list-message');
const formMessage = document.querySelector('#form-message');
const submitButton = form.querySelector('button[type="submit"]');
const apiUrl = 'http://localhost:8088/api/documents';

// Construire une ligne à partir des données renvoyées par l'API.
function createDocument(documentData) {
  const article = document.createElement('article');
  article.className = 'document-item';

  const icon = document.createElement('span');
  icon.className = 'document-icon doc-icon';
```

```
  icon.setAttribute('aria-hidden', 'true');
  icon.textContent = 'DOC';

  const details = document.createElement('div');
  details.className = 'document-details';

  const heading = document.createElement('h3');
  heading.textContent = documentData.title;

  const metadata = document.createElement('p');
  metadata.textContent = documentData.category;

  const categoryLabel = document.createElement('span');
  categoryLabel.className = 'category-label';
  categoryLabel.textContent = documentData.category;

  details.append(heading, metadata);
  article.append(icon, details, categoryLabel);
  return article;
}

// Charger la liste depuis Laravel et afficher les données reçues.
async function loadDocuments() {
  listMessage.textContent = 'Chargement des documents…';

  try {
    const response = await fetch(apiUrl, {
      headers: { Accept: 'application/json' },
    });
    if (!response.ok) {
      throw new Error('La liste est indisponible.');
    }

    const documents = await response.json();
    documentList.replaceChildren();
    documents.forEach((documentData) => {
      documentList.append(createDocument(documentData));
    });
    documentCount.textContent = `${documents.length} document${documents.length > 1 ? 's' : ''}`;
    listMessage.textContent = documents.length === 0 ? 'Aucun document pour le moment.' : '';
  } catch (error) {
    documentList.replaceChildren();
    documentCount.textContent = 'Indisponible';
    listMessage.textContent = 'Impossible de charger les documents. Vérifiez que Laravel fonctionne, puis rechargez la
page.';
  }
}

function showFormMessage(message, success) {
  formMessage.textContent = message;
  formMessage.classList.remove('d-none', 'alert-success', 'alert-danger');
  formMessage.classList.add(success ? 'alert-success' : 'alert-danger');
}

form.addEventListener('submit', async (event) => {
  event.preventDefault();

  const title = titleInput.value.trim();
  titleInput.setCustomValidity(title.length < 3 ? 'Saisissez au moins 3 caractères.' : '');
  form.classList.add('was-validated');
  formMessage.classList.add('d-none');

  if (!form.checkValidity()) {
    form.querySelector(':invalid')?.focus();
    return;
  }

  submitButton.disabled = true;
```

```
  let errorMessage = 'Impossible de joindre le serveur. Vérifiez la connexion puis réessayez.';
  try {
    const response = await fetch(apiUrl, {
      method: 'POST',
      headers: {
        Accept: 'application/json',
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ title, category: categoryInput.value }),
    });

    if (response.status === 422) {
      errorMessage = 'Le titre ou la catégorie est invalide.';
      throw new Error(errorMessage);
    }
    if (!response.ok) {
      errorMessage = 'Impossible d’ajouter le document. Vérifiez que Laravel fonctionne.';
      throw new Error(errorMessage);
    }

    form.reset();
    form.classList.remove('was-validated');
    titleInput.setCustomValidity('');
    showFormMessage('Document enregistré.', true);
    await loadDocuments();
  } catch (error) {
    showFormMessage(errorMessage, false);
  } finally {
    submitButton.disabled = false;
  }
});

titleInput.addEventListener('input', () => titleInput.setCustomValidity(''));

// Refermer le menu mobile après un clic sur un lien.
document.querySelectorAll('#navigation .nav-link').forEach((link) => {
  link.addEventListener('click', () => {
    const navigation = document.querySelector('#navigation');
    if (navigation.classList.contains('show')) {
      bootstrap.Collapse.getOrCreateInstance(navigation).hide();
    }
  });
});

loadDocuments();
```

La fonction `loadDocuments` est appelée au chargement et après une création réussie. Si la création réussit mais que le rechargement de la liste échoue, le message « Document enregistré » reste affiché et la zone de liste indique son erreur. La préférence d'affichage continue d'être gérée par `assets/js/securite.js`.

7. Démarrer et vérifier

Assurez-vous que les conteneurs Laravel sont démarrés et que les migrations ont été exécutées dans la base de formation. Dans le dossier front-end, ouvrez **`index.html` directement dans votre navigateur**.

1. La liste présente les documents déjà enregistrés, ou indique qu'elle est vide.
2. Saisissez un titre et une catégorie, puis cliquez sur **Ajouter le document**.
3. Le message « Document enregistré » apparaît et le document rejoint la liste.
4. Rechargez la page : le document est toujours présent, car Laravel l'a enregistré dans PostgreSQL.
5. Activez **Affichage compact** puis rechargez : la préférence reste mémorisée si le navigateur autorise `localStorage`.

6. Essayez un titre vide ou trop court : le navigateur doit signaler l'erreur avant l'envoi.

À la fin du TD, si vous souhaitez arrêter Laravel, ouvrez le terminal du projet Laravel et exécutez `docker compose stop`. Cette commande conserve les données du volume PostgreSQL.

8. Comparer avec la correction

La branche `jour2-fetch-documents` contient le résultat attendu. Ne changez pas de branche sans avoir sauvegardé votre travail : les fichiers modifiés peuvent empêcher Git de changer de branche. Pour consulter la correction depuis votre branche :

```
git fetch origin
git diff origin/jour2-fetch-documents -- index.html assets/js/app.js
```

Questions de compréhension

1. Quelle méthode HTTP charge la liste ? Quelle méthode ajoute un document ?
2. Pourquoi vérifie-t-on `response.ok` avant d'utiliser la réponse ?
3. À quoi sert `JSON.stringify` dans la requête POST ?
4. Que se passe-t-il si Laravel ne peut pas être joint ?
5. Qu'est-ce qui est conservé dans PostgreSQL et qu'est-ce qui reste dans `localStorage` ?

En cas de difficulté

Situation	Vérification
La page affiche une erreur de chargement ou d'ajout	Dans le projet Laravel, vérifiez `docker compose ps`, les routes CRUD et les migrations. L'API doit répondre sur `http://localhost:8088/api/documents`.
Le navigateur signale une erreur CORS	Vérifiez la configuration CORS de Laravel et consultez la console. Le comportement des pages `file://` peut varier selon le navigateur ; demandez au formateur si le poste bloque ces requêtes.
La liste ne change pas après modification du code	Enregistrez les fichiers et rechargez `index.html`. Vérifiez la console du navigateur.
Une modification est refusée par Git	Sauvegardez votre travail et demandez l'aide du formateur avant de changer de branche.